

Sila Contract Security Verification Standard 0.0.1

Bleeding Edge Version

SILA IDENTITY EDITION - 2026

Adapted from OWASP SCSVS 0.0.1 (2024) under CC BY-SA 4.0.

This derivative is not an official OWASP publication or certification.



2024

Contents

Frontispiece	6
About the Standard	6
Copyright and License	6
Project Leads	6
Major Contributors and Reviewers	6
Major Supporter and Sponsor	7
CredShields	7
Preface	8
Scope and Purpose	8
A Collaborative Effort	9
Looking Ahead	9
Utilizing the SCSVS	10
Security Verification Layers	10
Assumptions	10
Assessment and Certification	12
OWASP's Stance on SCSVS Certifications and Trust Marks	12
Guidance for Certifying Organizations	12
Certification Reports	12
S1. Architecture, Design, and Threat Modeling	14
S1.1 Secure Design Patterns	14
Control Objective	14
S1.1.B Separation of Concerns	14
S1.2 Proxy Patterns	15
Control Objective	15
S1.2.B Managing Deprecation	16
S1.2.C Transparent vs. Opaque Proxies	16
S1.3 Threat Modeling	17
Control Objective	17
S1.3.B Assessing Risks	17
S1.3.C Implementing Mitigations	17
S2. Policies, Procedures, and Code Management	18
Control Objective	18
S2.1 Development Policies	18
Control Objective	18
S2.1.A Secure Coding Standards	18
S2.1.B Code Review Processes	18

S2.2 Code Clarity	19
Control Objective	19
S2.2.A Readability and Documentation	19
S2.2.B Code Linting and Formatting Tools	20
S2.3 Test Coverage	20
Control Objective	20
S2.3.A Unit Tests, Integration Tests, Automated Testing	20
S2.3.B Security-Specific Tests	21
S3. Business Logic and Economic Security	22
Control Objective	22
S3.1 Economic Models	22
Control Objective	22
S3.1.A Incentive Structures	22
S3.2 Tokenomics	23
Control Objective	23
S3.2.A Economic Security of Tokens and Their Use Cases	23
S3.3 Preventing Reentrancy and Logic Flaws	23
Control Objective	23
S3.3.A Transaction Flow Security	23
S3.3.B Function Integrity	24
S4. Access Control and Authentication	25
Control Objective	25
S4.1 Role-Based Access Control (RBAC)	25
Control Objective	25
S4.1.A Multi-Signature Schemes	25
S4.1.B Identity Verification	25
S4.1.C Least Privilege Principle	26
S4.2 Authorization Mechanisms	26
Control Objective	26
S4.2.A Secure Access to Critical Functions	26
S4.2.B Timed Permissions	27
S4.3 Decentralized Identity	27
Control Objective	27
S4.3.A Decentralized Identifiers (DIDs)	27
S4.3.B Verifiable Credentials	28
S5. Secure Interactions and Communications	29
Control Objective	29
S5.1 Contract Interactions	29
Control Objective	29

S5.1.A Secure Message Passing	29
S5.1.B Minimal Trusted Surface	30
S5.2 Oracle Integrations	30
Control Objective	30
S5.2.A Secure Data Feeds	30
S5.2.B Decentralized Oracles	31
S5.3 Cross-Chain Interactions	31
Control Objective	31
S5.3.A Handling External Calls Securely	32
S5.3.B Atomic Swaps	32
S5.4 Bridges	32
Control Objective	32
S5.4.A Cross-Chain Transaction Security	33
S6. Cryptographic Practices	34
Control Objective	34
S6.1 Key Management	34
Control Objective	34
S6.1.A Secure Handling and Storage of Private Keys	34
S6.1.B Multi-Signature Wallets	35
S6.2 Signature Verification	35
Control Objective	35
S6.2.A Cryptographic Techniques for Authentication	35
S6.2.B Standard Compliance (e.g., SIP-712)	35
S6.3 Secure Random Number Generation	36
Control Objective	36
S6.3.A Best Practices for Randomness	36
S7. Arithmetic and Logic Security	37
Control Objective	37
S7.1 Preventing Overflow/Underflow	37
Control Objective	37
S7.1.A Use of Safe Math Libraries	37
S7.1.B Fixed-Point Arithmetic	38
S7.2 Arithmetic Integrity	38
Control Objective	38
S7.2.A Secure Calculations and Logic	38
S7.2.B Precondition and Postcondition Checks	39
S8. Denial of Service (DoS)	40
Control Objective	40

S8.1 Gas Limits	40
Control Objective	40
S8.1.A Efficient Loop and Function Design	40
S8.1.B Fallback Mechanisms	40
S8.2 Resilience Against Resource Exhaustion	41
Control Objective	41
S8.2.A Rate Limiting	41
S9. Blockchain Data and State Management	42
Control Objective	42
S9.1 State Management	42
Control Objective	42
S9.1.A Efficient and Secure State Handling	42
S9.1.B State Channels	42
S9.2 Data Privacy	43
Control Objective	43
S9.2.A Ensuring Sensitive Data is Secure	43
S9.2.B Zero-Knowledge Proofs	43
S9.2.C Private Transactions	43
S9.2.D Confidential Contracts	44
S9.3 Event Logging	44
Control Objective	44
S9.3.A Transparent and Secure Logging Practices	44
S9.3.B Log Analysis	45
S9.4 Decentralized Storage	45
Control Objective	45
S9.4.A IPFS, Arweave	45
S10. Gas Usage, Efficiency, and Limitations	46
Control Objective	46
S10.1 Optimizing Gas Usage	46
Control Objective	46
S10.1.A Understanding Gas Costs and Limits	46
S10.1.B Gas Estimation Tools	46
S10.2 Efficient Contract Design	46
Control Objective	46
S10.2.A Layer 2 Solutions	47
S11. Component-Specific Security	48
Control Objective	48
S11.1 Tokens (SRC20, SRC721, SRC1155)	48
Control Objective	48

S11.1.A Secure Implementation and Management	48
S11.2 NFT Security	49
Control Objective	49
S11.2.A Best Practices for Non-Fungible Tokens	49
S11.3 Vaults	49
Control Objective	49
S11.3.A Secure Asset Storage and Management	49
S11.4 Liquid Staking	50
Control Objective	50
S11.4.A Secure Staking Mechanisms	50
S11.5 Liquidity Pools (AMMs)	50
Control Objective	50
S11.5.A Security in Automated Market Makers	50
S11.6 Uniswap V4 Hook	50
Control Objective	50
S11.6.A Secure Integration and Customization	51
Appendix A: Glossary	52
Appendix B: References	55
OWASP Core Projects	55

Frontispiece

About the Standard

The Sila Contract Security Verification Standard (SCSVS) is a list of specific security requirements or tests for smart contracts, primarily written in Solidity and deployed on SVM-based blockchains. These requirements are intended to be used by architects, developers, testers, security professionals, tool vendors, and consumers to define, build, test, and verify secure smart contracts, decentralized applications (dApps) and blockchain protocols. The standard promotes best practices for ensuring the security and integrity of smart contracts and decentralized finance (DeFi) systems.

Copyright and License

Version 0.0.1 (Bleeding Edge version), 2024



Figure 1: license

Copyright © 2008-2024 The OWASP Foundation. This document is released under the Creative Commons Attribution-ShareAlike 4.0 International License. For any reuse or distribution, you must make clear to others the license terms of this work.

Project Leads

Shashank (CredShields)

Major Contributors and Reviewers

Pratik Lagaskar

Nehal Pillai

Aditya Dixit

If a credit is missing from the 0.0.1 credit list above, please log a ticket at GitHub to be recognized in future 0.x updates.

The Sila Contract Security Verification Standard (SCSVS) is built upon the initial research performed into smart contract security by various blockchain security experts. Much of the concept, structure, boilerplate, and tooling for the SCSVS has been adapted from the OWASP ASVS project. We extend our gratitude to all those previously involved in the OWASP ASVS for their contributions.

Major Supporter and Sponsor

This initiative would not have been possible without the support of our sponsor and the resources they have provided. We would like to express our gratitude to the following for their support.

CredShields

The OWASP SCSVS project was initiated to share the knowledge gained from the CredShields Security Team's research into Smart Contract security while developing SolidityScan.com, an AI-powered vulnerability scanner for Smart Contracts. We extend our gratitude to CredShields for their efforts in defining the initial requirements and founding this project.

Preface

Welcome to the alpha release of the Sila Contract Security Verification Standard (SCSVS), which serves as a framework for assessing the security of smart contracts built on Sila Virtual **Machine (SVM)-based blockchains**, specifically those developed using **Solidity**.

Smart contracts are autonomous programs that execute on decentralized blockchain networks, facilitating a wide range of applications, including decentralized finance (DeFi), governance systems, and tokenized assets. However, the immutability and high-value nature of blockchain ecosystems introduce unique risks and challenges. This makes security in smart contract development not only critical but also highly specialized.

The SCSVS aims to provide **comprehensive, actionable guidelines** that support developers, auditors, security professionals, and architects in building and maintaining secure smart contracts, particularly within the Solidity ecosystem on SVM-based blockchains. It seeks to address common and emerging vulnerabilities, such as **reentrancy attacks**, **integer overflows/underflows**, **gas optimization issues**, and **economic attacks**—all of which pose significant risks to smart contract security and user trust.

This alpha release is the result of a collaborative effort by professionals and experts across various sectors, including blockchain security, financial technology, and decentralized application (dApp) development. The SCSVS is designed to offer **flexible and evolving guidance** for securing smart contracts, addressing both **functional** and **non-functional security aspects**.

Scope and Purpose

The SCSVS provides detailed verification requirements that focus on the **design, implementation, and testing** phases of smart contract development. It seeks to guide stakeholders through:

- **Designing with security in mind:** Ensuring that security is a core principle during the planning stages of smart contract development.
- **Implementing secure coding practices:** Emphasizing Solidity-specific security measures to mitigate risks inherent to the SVM environment.
- **Auditing and Testing:** Offering best practices for conducting rigorous security audits, penetration testing, and ongoing monitoring of smart contracts once deployed.

This standard is particularly relevant for developers who work on **DeFi protocols**, **token contracts**, **decentralized exchanges (DEXs)**, and any application that interacts with assets or governance in a decentralized manner. Its guidelines are aligned with the broader needs of the Sila and SVM-based blockchain ecosystems, though many principles apply to other smart contract platforms as well.

A Collaborative Effort

The security challenges facing smart contract developers are constantly evolving, as adversaries seek new ways to exploit weaknesses in decentralized systems. The **SCSVS alpha release** is designed to be a starting point, and we openly invite contributions from the community to help expand, refine, and adapt these guidelines.

We understand that no security standard can be entirely comprehensive, especially in the dynamic field of blockchain technology, which is rapidly advancing. The aim is to foster **collaboration** and **continuous improvement**. Your feedback and active participation will be invaluable in ensuring that the SCSVS remains practical, effective, and up to date with emerging threats and technologies.

Looking Ahead

The Sila Contract Security Verification Standard is not a final document. This alpha release is the foundation for a **living standard** that will grow and adapt with the needs of the community and advances in smart contract development. We encourage the community to engage actively with this project—whether by contributing ideas, identifying gaps, or proposing enhancements.

In the spirit of OWASP’s mission, this standard seeks to improve the security posture of the smart contract ecosystem, safeguarding both developers and users alike. We sincerely thank all contributors, and we look forward to your continued support in shaping the future of secure smart contract development.

Together, we can build a safer decentralized future.

Utilizing the SCSVS

The Sila Contract Security Verification Standard (SCSVS) serves several key purposes:

- **Assisting Development Teams:** Guide smart contract developers in designing, implementing, and maintaining secure decentralized applications (dApps) and contracts, particularly on SVM-based blockchains.
- **Framework for Security Teams:** Assist security professionals in setting requirements, conducting security audits, and performing penetration tests against smart contract systems to ensure they meet robust security standards.
- **Aligning Security Benchmarks:** Establish a common security framework that can be adopted by blockchain platforms, vendors, developers, and clients regarding security expectations in smart contracts and decentralized applications.

Security Verification Layers

The SCSVS categorizes security verification into three distinct levels, each aimed at different levels of security assurance in smart contract development and deployment:

1. **SCSVS Level 1 - Basic Security:** This level is designed for smart contracts with lower security risks. It focuses on fundamental security controls, ensuring baseline protection for any decentralized application.
2. **SCSVS Level 2 - Moderate Security:** Ideal for smart contracts that handle sensitive data, financial transactions, or are part of a DeFi ecosystem. Level 2 provides a more balanced approach to security, addressing common vulnerabilities like reentrancy attacks, gas inefficiencies, and access control weaknesses.
3. **SCSVS Level 3 - High Assurance Security:** This level is tailored for mission-critical smart contracts where significant financial assets, governance, or high-value transactions are at stake. Level 3 ensures extensive security measures and covers advanced protections such as formal verification, multi-signature wallets, and decentralized governance.

Each level of the SCSVS provides a detailed set of security requirements, mapping these to essential security features and practices needed to build secure smart contracts. Whether developing, auditing, or deploying smart contracts, the SCSVS offers a clear roadmap to help teams at every stage.

Assumptions

When utilizing the SCSVS, it's important to consider the following assumptions:

- The SCSVS is not a replacement for standard secure development practices such as **secure coding** or following a **Secure Software Development Life Cycle (SSDLC)**. It should complement

these practices by addressing specific security needs for **SVM-based smart contracts** and decentralized applications.

- The SCSVS is not intended to replace comprehensive **threat modeling** or **security reviews**. It serves as a specialized guide to help identify and mitigate vulnerabilities unique to smart contracts. Employing the SCSVS should enhance, not replace, traditional security risk assessments and penetration tests.

While the SCSVS offers a solid framework for improving the security of smart contracts, it cannot ensure complete security on its own. It should be considered a foundational security standard, with additional protective measures added as necessary to address specific vulnerabilities and evolving threats in decentralized environments.

Assessment and Certification

OWASP's Stance on SCSVS Certifications and Trust Marks

OWASP, as a vendor-neutral not-for-profit organization, does not currently certify any vendors, verifiers, or smart contracts.

All such assurance assertions, trust marks, or certifications are not officially vetted, registered, or certified by OWASP. Therefore, organizations relying on third-party verification or certifications must carefully evaluate the trust placed in any external entity or trust mark claiming **Smart Contract Security Verification Standard (SCSVS)** certification.

This should not discourage organizations from offering security verification or audit services, as long as they do not claim to provide official OWASP certification.

Guidance for Certifying Organizations

For Sila Contract Security Verification Standard (SCSVS) compliance, an “open book” review is recommended, where assessors are granted access to essential resources such as smart contract developers, project documentation, source code, and authenticated blockchain interfaces (including access to the blockchain explorer, transaction logs, and testing environments). It's essential that assessors gain access to at least one account for each user role, particularly if the contract supports permissioned or role-based access.

It is important to note that the SCSVS only covers the security requirements specific to **SVM-based smart contracts** and does not extend to general application security controls (e.g., web interfaces, databases, or other non-blockchain components). Any additional systems or non-blockchain elements should be verified against appropriate security standards, such as the OWASP ASVS.

Certification Reports

Certification reports should clearly define the scope of the verification, specifying which smart contracts, components, or decentralized applications (dApps) were reviewed, and should list any excluded items from the review. The report should summarize the findings, detailing both passed and failed security controls, alongside guidance on how to remediate any failures.

Industry-standard practices for security certification require thorough documentation of the verification process. This should include:

- **Work papers:** Notes and records on each step of the verification process.
- **Screenshots:** Evidence of security control tests, such as transaction hashes or audit results.
- **Scripts:** Used for testing and replication of discovered issues.
- **Blockchain logs:** Detailed records of the verification process including contract interactions, transactions, and gas usage.

Automated tools alone are insufficient to verify SCSVS compliance. All verification reports must provide conclusive, manually validated evidence that demonstrates the thorough and accurate testing of all required controls. In case of disputes, documentation should include adequate evidence to confirm that each control has been appropriately tested and validated.

S1. Architecture, Design, and Threat Modeling

S1.1 Secure Design Patterns

Control Objective

Ensure that smart contracts are designed with modularity, upgradability, and separation of concerns to enable secure operations, upgrades, and maintenance. Contracts should be designed to minimize security risks related to complex upgrades, privilege transfers, and mismanagement of dependencies. ### Security Verification Requirements ### S1.1.A Modularity and Upgradability

Ref	Requirement	L1	L2	L3	SWE
S1.1.A1	Verify that the contract is divided into modular components or contracts.		✓	✓	
S1.1.A2	Ensure that upgrade mechanisms are designed to allow secure and controlled updates.		✓	✓	
S1.1.A3	Check that module boundaries are clearly defined and that dependencies are managed.		✓	✓	
S1.1.A4	Ensure that changes to storage variable order or types between contract versions are managed to avoid storage collisions and data corruption.		✓	✓	
S1.1.A5	Verify that critical privilege transfers are conducted in a two-step process to ensure secure and reliable privilege changes.			✓	
S1.1.A6	Verify that the data location of parameters and return variables is correctly handled when overriding internal and public functions to avoid generating invalid code during virtual function calls.			✓	

S1.1.B Separation of Concerns

Ref	Requirement	L1	L2	L3	SWE
S1.1.B1	Verify that different functionalities are separated into distinct contracts or modules.		✓	✓	
S1.1.B2	Ensure that each module has a single responsibility and minimal dependencies on other modules.		✓	✓	
S1.1.B3	Check for any cross-module dependencies that could lead to security risks.		✓	✓	

Ref	Requirement	L1	L2	L3	SWE
S1.1.B4	Ensure that the protocol maintains consistent and reliable operation during the transfer of privileges, with considerations for various edge cases.			✓	
S1.1.B5	Verify that proxy contracts use the <code>onlyInitializing</code> modifier instead of <code>initializer</code> to ensure proper initialization.			✓	
S1.1.B6	Verify that storage layouts between contract versions are consistent to prevent data corruption and unpredictable behavior.			✓	
S1.1.B7	Ensure that immutable variables are consistent across implementations during proxy upgrades.			✓	
S1.1.B8	Verify that implementations of the same logic across different parts of the contract are consistent to avoid introducing errors or vulnerabilities.			✓	
S1.1.B9	Ensure that SIL and WSIL are handled separately with appropriate checks to avoid errors due to incorrect assumptions about exclusivity.			✓	
S1.1.B10	Verify that contracts with constructors are not used in a proxy setup, and <code>initializer</code> logic is used instead.			✓	

S1.2 Proxy Patterns

Control Objective

Ensure that proxy patterns and upgrade mechanisms are implemented securely and managed properly, to mitigate risks during contract upgrades, deprecations, and transitions between contract versions. ### Security Verification Requirements ### S1.2.A Upgrade Mechanisms

Ref	Requirement	L1	L2	L3	SWE
S1.2.A1	Verify that an upgrade mechanism (e.g., proxy pattern) is implemented for the contract.		✓	✓	
S1.2.A2	Ensure that the upgrade process includes safeguards against unauthorized upgrades.		✓	✓	
S1.2.A3	Check that the upgrade mechanism is documented and reviewed for security.		✓	✓	
S1.2.A4	Verify that immutable variables are consistent across implementations during proxy upgrades to prevent misuse.			✓	

Ref	Requirement	L1	L2	L3	SWE
S1.2.A5	Verify that <code>selfdestruct</code> and <code>delegatecall</code> in implementation contracts do not introduce vulnerabilities or unexpected behaviors in a proxy setup.			✓	
S1.2.A6	Verify that <code>UUPSUpgradeable</code> contracts are protected against vulnerabilities that may affect uninitialized implementation contracts, ensuring secure upgrade mechanisms.			✓	

S1.2.B Managing Deprecation

Ref	Requirement	L1	L2	L3	SWE
S1.2.B1	Verify that deprecated contract versions are correctly marked and handled.			✓	
S1.2.B2	Ensure that access to deprecated versions is restricted or disabled.			✓	
S1.2.B3	Check that migration paths from deprecated versions to new versions are secure.			✓	

S1.2.C Transparent vs. Opaque Proxies

Ref	Requirement	L1	L2	L3	SWE
S1.2.C1	Verify whether a transparent or opaque proxy pattern is used and its suitability for the contract.		✓	✓	
S1.2.C2	Ensure that the proxy pattern is correctly implemented and does not introduce vulnerabilities.		✓	✓	
S1.2.C3	Check that the proxy pattern's choice is documented and justified.		✓	✓	
S1.2.C4	Ensure that uninitialized contracts cannot be taken over by attackers and that initialization functions are secured with the correct modifiers.			✓	
S1.2.C5	Verify that <code>TransparentUpgradeableProxy</code> and similar proxy patterns handle selector clashes and non-decodable calldata correctly to maintain transparency.			✓	

S1.3 Threat Modeling

Control Objective

Identify, assess, and mitigate security threats for smart contract systems by implementing a thorough threat modeling process, ensuring that risks are minimized and protections are in place for critical contract features. ### Security Verification Requirements ### S1.3.A Identifying Threats

Ref	Requirement	L1	L2	L3	SWE
S1.3.A1	Verify that potential threats are identified and documented.	✓	✓	✓	
S1.3.A2	Ensure that the threat identification process includes input from security experts.		✓	✓	
S1.3.A3	Check that threats are categorized based on their impact and likelihood.		✓	✓	
S1.3.A4	Implement protections against front-running in governor proposal creation to prevent attackers from blocking proposals or gaining undue advantages.			✓	

S1.3.B Assessing Risks

Ref	Requirement	L1	L2	L3	SWE
S1.3.B1	Verify that risk assessments are performed for identified threats.		✓	✓	
S1.3.B2	Ensure that risks are prioritized based on their potential impact and likelihood.		✓	✓	
S1.3.B3	Check that risk assessment results are documented and reviewed.		✓	✓	

S1.3.C Implementing Mitigations

Ref	Requirement	L1	L2	L3	SWE
S1.3.C1	Verify that mitigations are implemented for high-priority risks.		✓	✓	
S1.3.C2	Ensure that mitigation strategies are documented and tested.		✓	✓	
S1.3.C3	Check that the effectiveness of implemented mitigations is reviewed and validated.		✓	✓	

S2. Policies, Procedures, and Code Management

Control Objective

Ensure that development policies and procedures are in place to promote secure coding practices, thorough code reviews, and comprehensive testing. The aim is to prevent vulnerabilities and enhance the maintainability and clarity of smart contract code.

S2.1 Development Policies

Control Objective

Establish and enforce secure coding standards and review processes to minimize vulnerabilities and ensure best practices are followed throughout the development lifecycle.

S2.1.A Secure Coding Standards

Ref	Requirement	L1	L2	L3	SWE
S2.1.A1	Ensure that developers do not use outdated compiler versions and adhere to the latest compiler recommendations.		✓	✓	
S2.1.A2	Verify that deprecated functions are not used in the code.		✓	✓	

S2.1.B Code Review Processes

Ref	Requirement	L1	L2	L3	SWE
S2.1.B1	Verify that all smart contract code changes are reviewed by at least two independent developers with expertise in smart contract security before merging to the main branch.		✓	✓	
S2.1.B2	Ensure that code reviews of smart contracts involve automated static analysis tools specifically designed for smart contracts, and that all flagged issues are addressed or documented prior to merging.		✓	✓	

Ref	Requirement	L1	L2	L3	SWE
S2.1.B3	Check that the code review process for smart contracts includes a thorough analysis for vulnerabilities such as reentrancy attacks, integer overflows, and improper access control.		✓	✓	
S2.1.B4	Verify that code reviews include adherence to smart contract development standards, such as the use of safe math libraries and secure design patterns.		✓	✓	
S2.1.B5	Ensure that code reviews incorporate a checklist of common smart contract vulnerabilities, and that each item on the list is addressed before code is approved.		✓	✓	

S2.2 Code Clarity

Control Objective

Promote code clarity and maintainability through thorough documentation, logical structure, and adherence to consistent coding standards, enabling easier understanding and modification by developers.

S2.2.A Readability and Documentation

Ref	Requirement	L1	L2	L3	SWE
S2.2.A1	Ensure that all smart contract functions and critical code blocks are documented with clear comments that explain their purpose and logic.		✓	✓	
S2.2.A2	Verify that the structure of the smart contract is logical and organized to facilitate understanding and modification by other developers.		✓	✓	
S2.2.A3	Check that the smart contract documentation includes a high-level overview of its functionality, usage instructions, and any dependencies on other contracts or systems.		✓	✓	

Ref	Requirement	L1	L2	L3	SWE
S2.2.A4	Ensure that smart contract code follows consistent naming conventions for variables, functions, and contract names to improve readability and maintainability.		✓	✓	

S2.2.B Code Linting and Formatting Tools

Ref	Requirement	L1	L2	L3	SWE
S2.2.B1	Ensure that a code linting tool specific to smart contracts is integrated into the development workflow to enforce coding standards and style guidelines.		✓	✓	
S2.2.B2	Verify that all smart contract code passes linting checks without errors or warnings before being committed to the repository.		✓	✓	
S2.2.B3	Check that code formatting tools are applied to maintain consistent indentation, spacing, and line breaks in smart contract code.		✓	✓	
S2.2.B4	Ensure that the linting and formatting configurations are reviewed and updated regularly to reflect new best practices and emerging issues in smart contract development.		✓	✓	
S2.2.B5	Verify that the linting and formatting tools are compatible with the development environment and do not introduce unintended changes to the smart contract code.		✓	✓	

S2.3 Test Coverage

Control Objective

Ensure comprehensive test coverage for smart contracts, encompassing unit tests, integration tests, and security-specific tests, to identify vulnerabilities and maintain code quality throughout development.

S2.3.A Unit Tests, Integration Tests, Automated Testing

Ref	Requirement	L1	L2	L3	SWE
S2.3.A1	Verify that all critical functions in the smart contract have comprehensive unit tests that cover both typical and edge cases.		✓	✓	
S2.3.A2	Ensure that integration tests are implemented to validate the interactions between the smart contract and other contracts or external systems.		✓	✓	
S2.3.A3	Check that automated tests are set up to run on each code commit to detect regressions and maintain continuous quality of the smart contract.		✓	✓	
S2.3.A4	Verify that test coverage tools are used to monitor and achieve a high percentage of coverage for the smart contract code.		✓	✓	
S2.3.A5	Ensure that the testing framework supports mocking and simulating external dependencies to effectively isolate and test specific functionalities of the smart contract.		✓	✓	

S2.3.B Security-Specific Tests

Ref	Requirement	L1	L2	L3	SWE
S2.3.B1	Verify that the test suite for the smart contract includes security-specific tests designed to identify vulnerabilities such as reentrancy, integer overflows, and improper access controls.		✓	✓	
S2.3.B2	Ensure that the security tests validate proper implementation of access controls and permissions within the smart contract.		✓	✓	
S2.3.B3	Check that fuzz testing is conducted to uncover unexpected behaviors and potential security issues under various input scenarios.		✓	✓	
S2.3.B4	Verify that the smart contract's response to invalid inputs and edge cases is thoroughly tested to ensure robust security measures are in place.		✓	✓	

S3. Business Logic and Economic Security

Control Objective

Ensure that the smart contract's business logic and economic security are resilient against threats related to incentive structures, tokenomics, and logic vulnerabilities. Contracts should prevent abuse, misbehavior, or unexpected behaviors by implementing secure economic models, token handling, and transaction integrity.

S3.1 Economic Models

Control Objective

Ensure that economic models, including incentive structures and tokenomics, are designed and implemented in a way that secures value and incentivizes proper behavior within the ecosystem. Contracts should handle fluctuating token values and avoid creating opportunities for exploitation.

S3.1.A Incentive Structures

Ref	Requirement	L1	L2	L3	SWE
S3.1.A1	Ensure that the withdrawal process implements a pull-based approach rather than a push-based one to track accounting and allow users to pull payments.	✓	✓	✓	
S3.1.A2	The rate of cbSIL to SIL can decrease, impacting users who hold or interact with cbSIL. Ensure mechanisms are in place to handle fluctuations in conversion rates.		✓	✓	
S3.1.A3	Validators on the Sila 2.0 Beacon Chain can be penalized or slashed for misbehavior, which can affect the value of rSIL. Ensure that these dynamics are considered in value assessments and interactions.		✓	✓	
S3.1.A4	The conversion rate between SIL and rSIL might change over time based on the rewards accrued from staking. Ensure that these fluctuations are properly managed and captured.		✓	✓	

S3.2 Tokenomics

Control Objective

Ensure that tokens used within the smart contract ecosystem are securely implemented, including aspects such as value management, rebasing mechanisms, and reward systems. Contracts should prevent token vulnerabilities like double-spending, incorrect rewards, and improper fee handling.

S3.2.A Economic Security of Tokens and Their Use Cases

Ref	Requirement	L1	L2	L3	SWE
S3.2.A1	Ensure that Merkle trees do not contain duplicate proofs, as this can lead to vulnerabilities like double-spending.		✓	✓	
S3.2.A2	Verify that DeFi protocols account for tokens with negative rebase mechanisms, ensuring that value changes and potential miscalculations are properly handled and mitigated.		✓	✓	
S3.2.A3	Verify that reward claims are correctly implemented to ensure users receive their correct rewards.		✓	✓	
S3.2.A4	Verify that tokens do not have vulnerabilities such as incorrect fee application or unexpected behavior due to token transfer issues.		✓	✓	
S3.2.A5	Verify that all claimable addresses are included in the hashing process for Merkle tree leaves to prevent attackers from claiming funds they should not.		✓	✓	

S3.3 Preventing Reentrancy and Logic Flaws

Control Objective

Ensure the smart contract's transaction flow and logic integrity are protected from reentrancy attacks and logic flaws. Contracts should implement robust control structures and security patterns to prevent reentrancy, handle complex flows, and ensure that state transitions are secure and symmetrical.

S3.3.A Transaction Flow Security

Ref	Requirement	L1	L2	L3	SWE
S3.3.A1	Check for edge cases in loop control structures to prevent unexpected behaviors due to break or continue statements.		✓	✓	
S3.3.A2	Ensure that scenarios where sender and recipient are the same are considered to prevent unintended issues in smart contracts.		✓	✓	
S3.3.A3	Ensure that the <code>NonReentrant</code> modifier is applied before other modifiers in functions to prevent reentrancy attacks.		✓	✓	
S3.3.A4	Verify that the check-effect-interaction pattern is implemented to prevent reentrancy attacks.		✓	✓	
S3.3.A5	Ensure that function calls with arbitrary user input and low-level calls are handled securely to avoid introducing risks.		✓	✓	

S3.3.B Function Integrity

Ref	Requirement	L1	L2	L3	SWE
S3.3.B1	Ensure that functions intended to be unique per parameter set are not callable multiple times to prevent potential issues.		✓	✓	
S3.3.B2	Verify that state changes in functions, such as withdraw and deposit, are symmetrically handled to avoid undesired behavior due to inconsistencies.		✓	✓	

S4. Access Control and Authentication

Control Objective

Establish robust access control and authentication mechanisms to ensure that only authorized entities can perform sensitive operations within the smart contract. This includes implementing role-based access control (RBAC), secure authorization mechanisms, and decentralized identity management.

S4.1 Role-Based Access Control (RBAC)

Control Objective

Implement role-based access control to manage permissions and ensure that only authorized users can access specific functions. This includes validating identities, applying the least privilege principle, and ensuring appropriate access controls are in place.

S4.1.A Multi-Signature Schemes

Ref	Requirement	L1	L2	L3	SWE
S4.1.A1	Ensure that the visibility modifier for all functions is appropriate, preventing unnecessary exposure of internal functions.		✓	✓	

S4.1.B Identity Verification

Ref	Requirement	L1	L2	L3	SWE
S4.1.B1	Validate that unexpected addresses do not result in unintended behaviors, particularly when these addresses refer to contracts within the same protocol.		✓	✓	
S4.1.B2	Verify that functions like ecrecover handle all potential null addresses properly to avoid vulnerabilities arising from unexpected ecrecover outputs.		✓	✓	

S4.1.C Least Privilege Principle

Ref	Requirement	L1	L2	L3	SWE
S4.1.C1	Use msg.sender instead of tx.origin for authorization to avoid potential abuse from malicious contracts; include checks like require(tx.origin == msg.sender) to ensure the sender is an EOA.		✓	✓	
S4.1.C2	Certain addresses might be blocked or restricted from receiving tokens (e.g., LUSD). Ensure that address restrictions are properly managed and verified.		✓	✓	
S4.1.C3	Ensure that Guard's hooks (e.g., checkTransaction(), checkAfterExecution()) are executed to enforce critical security checks.		✓	✓	
S4.1.C4	Ensure that access controls are implemented correctly to determine who can use certain functions, and avoid unauthorized changes or withdrawals.		✓	✓	

S4.2 Authorization Mechanisms**Control Objective**

Implement secure authorization mechanisms to safeguard critical functions and sensitive operations, ensuring only authorized entities can perform these actions.

S4.2.A Secure Access to Critical Functions

Ref	Requirement	L1	L2	L3	SWE
S4.2.A1	Verify that the contract uses msg.sender for authorization instead of tx.origin to avoid vulnerabilities related to contracts that forward calls from legitimate users.		✓	✓	
S4.2.A2	Implement and verify appropriate access controls for functions that modify contract state or perform sensitive operations to prevent unauthorized access.		✓	✓	

S4.2.B Timed Permissions

Ref	Requirement	L1	L2	L3	SWE
S4.2.B1	Ensure that msg.sender validation is properly implemented when using Merkle trees to maintain security and prevent unauthorized access.		✓	✓	
S4.2.B2	Use whitelisting to restrict interactions to a specific set of addresses, providing additional security against malicious actors.		✓	✓	
S4.2.B3	Ensure that functions modifying the contract state or accessing sensitive operations have proper access controls implemented.		✓	✓	

S4.3 Decentralized Identity**Control Objective**

Implement decentralized identity solutions to ensure secure and reliable identity verification and management while maintaining user privacy.

S4.3.A Decentralized Identifiers (DIDs)

Ref	Requirement	L1	L2	L3	SWE
S4.3.A1	Verify that the smart contract for handling DIDs adheres to the latest standards and best practices for decentralized identity management.		✓	✓	
S4.3.A2	Ensure that the DID management contract includes mechanisms to prevent unauthorized modifications and ensure the integrity of DID records.		✓	✓	
S4.3.A3	Check that DID documents managed by the smart contract are securely stored and can be retrieved in a decentralized manner.		✓	✓	
S4.3.A4	Verify that the smart contract supports reliable DID resolution and includes mechanisms for handling conflicts and updates.		✓	✓	
S4.3.A5	Ensure that the smart contract maintains the privacy and confidentiality of DID-related information throughout its lifecycle.		✓	✓	

S4.3.B Verifiable Credentials

Ref	Requirement	L1	L2	L3	SWE
S4.3.B1	Verify that the smart contract manages verifiable credentials in a way that ensures their authenticity and integrity through cryptographic proofs.		✓	✓	
S4.3.B2	Ensure that the issuance process of verifiable credentials by the smart contract includes proper identity verification and validation procedures.		✓	✓	
S4.3.B3	Check that the smart contract supports cryptographic proofs to verify the validity of credentials without disclosing sensitive information.		✓	✓	
S4.3.B4	Verify that the smart contract includes a secure process for revoking verifiable credentials when necessary.		✓	✓	
S4.3.B5	Ensure that the smart contract's handling of verifiable credentials complies with relevant standards and best practices for secure credential management.		✓	✓	

S5. Secure Interactions and Communications

Control Objective

Establish secure interaction protocols for smart contracts to ensure safe communication between contracts, external oracles, and cross-chain integrations. This includes managing contract interactions, securing oracle integrations, handling cross-chain interactions, and ensuring the security of bridges.

S5.1 Contract Interactions

Control Objective

Ensure that all interactions between contracts are secure, minimizing risks associated with external calls, maintaining a minimal trusted surface, and handling errors appropriately.

S5.1.A Secure Message Passing

Ref	Requirement	L1	L2	L3	SWE
S5.1.A1	Ensure that calls to inherited functions from LzApp use recommended approaches (e.g., <code>_lzSend</code>) to avoid vulnerabilities associated with direct calls to <code>lzEndpoint.send</code> .		✓	✓	
S5.1.A2	Ensure that when interacting with external contracts, the <code>msg.sender</code> remains consistent to avoid security issues related to unexpected changes in sender context.		✓	✓	
S5.1.A3	Manage untrusted external contract calls to prevent unexpected results such as multiple withdrawals or out-of-order events.		✓	✓	
S5.1.A4	Missing verification of interacting pools can introduce risks. Ensure that all pools are properly verified before interaction to prevent potential security issues.		✓	✓	
S5.1.A5	Verify that the low-level <code>.delegatecall()</code> is properly managed, acknowledging that it converts the return value to a Boolean without providing the execution outcome.		✓	✓	

S5.1.B Minimal Trusted Surface

Ref	Requirement	L1	L2	L3	SWE
S5.1.B1	Verify that the smart contract minimizes its trusted surface by only interacting with other contracts and systems through well-defined and limited interfaces.		✓	✓	
S5.1.B2	Ensure that the smart contract includes checks to validate the trustworthiness and authenticity of interacting parties before executing sensitive operations.		✓	✓	
S5.1.B3	Check that the smart contract's interactions are designed to avoid dependencies on external data or contracts that could compromise security.		✓	✓	
S5.1.B4	Verify that the contract handles failures or unexpected behaviors from external interactions gracefully to avoid cascading failures.		✓	✓	
S5.1.B5	Ensure that interactions with other contracts are monitored and audited to detect and address any unusual or unauthorized activities.		✓	✓	

S5.2 Oracle Integrations**Control Objective**

Ensure that oracle integrations provide secure, reliable, and tamper-proof data feeds while maintaining data integrity and handling failures appropriately.

S5.2.A Secure Data Feeds

Ref	Requirement	L1	L2	L3	SWE
S5.2.A1	Verify that the smart contract uses oracles that provide secure and tamper-proof data feeds, including checks for data integrity and authenticity.		✓	✓	
S5.2.A2	Ensure that the smart contract validates the data received from oracles to prevent malicious or incorrect data from affecting contract operations.		✓	✓	

Ref	Requirement	L1	L2	L3	SWE
S5.2.A3	Check that the smart contract includes fallback mechanisms in case of oracle failure or unreliable data.		✓	✓	
S5.2.A4	Verify that the integration with oracles follows best practices for data security, including encryption and secure communication channels.		✓	✓	
S5.2.A5	Ensure that the smart contract's oracle integration is designed to handle any potential discrepancies or conflicts in data from multiple sources.		✓	✓	

S5.2.B Decentralized Oracles

Ref	Requirement	L1	L2	L3	SWE
S5.2.B1	Verify that the smart contract uses decentralized oracles to enhance data reliability and prevent single points of failure or manipulation.		✓	✓	
S5.2.B2	Ensure that the smart contract includes mechanisms to validate the consensus or majority opinion of decentralized oracles before taking actions based on their data.		✓	✓	
S5.2.B3	Check that the smart contract accounts for potential latency or delays in data from decentralized oracles to maintain operational reliability.		✓	✓	
S5.2.B4	Verify that the smart contract includes checks to prevent manipulation or collusion among decentralized oracles.		✓	✓	
S5.2.B5	Ensure that the decentralized oracle integration adheres to standards for security and reliability in multi-oracle environments.		✓	✓	

S5.3 Cross-Chain Interactions

Control Objective

Ensure secure handling of external calls and atomic swaps during cross-chain interactions to maintain operational reliability and prevent fraud.

S5.3.A Handling External Calls Securely

Ref	Requirement	L1	L2	L3	SWE
S5.3.A1	Ensure that parameters for Chainlink VRF (Verifiable Random Function) are thoroughly validated to prevent the fulfillRandomWord function from returning incorrect values instead of reverting.		✓	✓	
S5.3.A2	Implement robust security checks for cross-chain messaging to ensure correct permissions and intended functionality.		✓	✓	
S5.3.A3	Verify that contracts created using the CREATE opcode handle block reorganizations securely to prevent unexpected eliminations.		✓	✓	
S5.3.A4	Ensure robust cross-chain messaging security checks to prevent replay attacks where signatures valid on one chain might be exploited on another.		✓	✓	

S5.3.B Atomic Swaps

Ref	Requirement	L1	L2	L3	SWE
S5.3.B1	Verify that the smart contract supports atomic swaps with robust mechanisms to ensure that swaps are completed successfully or not executed at all.		✓	✓	
S5.3.B2	Ensure that the smart contract includes checks to validate the atomic swap conditions and prevent partial or fraudulent swaps.		✓	✓	
S5.3.B3	Check that the smart contract handles potential failures or disputes in atomic swaps securely and fairly.		✓	✓	
S5.3.B4	Verify that the atomic swap functionality is tested thoroughly to cover various scenarios and edge cases.		✓	✓	

S5.4 Bridges**Control Objective**

Ensure the security of cross-chain transactions by implementing robust validation and verification mechanisms to prevent fraud and maintain data integrity.

S5.4.A Cross-Chain Transaction Security

Ref	Requirement	L1	L2	L3	SWE
S5.4.A1	Verify that the smart contract for cross-chain transactions includes robust mechanisms for verifying and validating transactions across different chains.		✓	✓	
S5.4.A2	Ensure that the smart contract prevents double-spending and replay attacks in cross-chain transactions by implementing appropriate security checks.		✓	✓	
S5.4.A3	Check that the cross-chain transaction contract handles communication and data integrity securely between different blockchain networks.		✓	✓	
S5.4.A4	Verify that the smart contract includes fallback and recovery mechanisms for handling failures or inconsistencies in cross-chain transactions.		✓	✓	

S6. Cryptographic Practices

Control Objective

Establish secure cryptographic practices for managing keys, verifying signatures, and generating random numbers to protect the integrity and authenticity of transactions and data within smart contracts.

S6.1 Key Management

Control Objective

Ensure secure handling and storage of private keys and implement robust signature verification processes to prevent unauthorized access and actions.

S6.1.A Secure Handling and Storage of Private Keys

Ref	Requirement	L1	L2	L3	SWE
S6.1.A1	Verify that the <code>ecrecover()</code> function handles malformed inputs correctly and does not return invalid data.		✓	✓	
S6.1.A2	Ensure that signature verification mechanisms are robust against signature malleability and replay attacks, particularly by using nonces or hashed messages rather than relying solely on the signature itself.		✓	✓	
S6.1.A3	Verify that <code>SignatureChecker.isValidSignatureNow</code> handles edge cases properly and does not revert unexpectedly, considering the ABI decoding issues introduced in Solidity 0.8.		✓	✓	
S6.1.A4	Ensure that all signatures are checked thoroughly to prevent unauthorized transactions or actions due to weak or improperly managed signature validation.		✓	✓	
S6.1.A5	Validate that signature protection mechanisms are up-to-date and resistant to signature malleability attacks by avoiding outdated libraries and practices.		✓	✓	

S6.1.B Multi-Signature Wallets

Ref	Requirement	L1	L2	L3	SWE
S6.1.B1	Verify that multi-signature wallets require a predefined number of signatures before executing any transaction.		✓	✓	
S6.1.B2	Ensure that the multi-signature wallet logic is resistant to replay attacks.		✓	✓	
S6.1.B3	Verify that the process of adding or removing signatories from the multi-signature wallet is secure and controlled.		✓	✓	

S6.2 Signature Verification**Control Objective**

Implement cryptographic techniques that ensure the secure verification of signatures and compliance with standards to maintain the integrity of authenticated transactions.

S6.2.A Cryptographic Techniques for Authentication

Ref	Requirement	L1	L2	L3	SWE
S6.2.A1	Ensure that cryptographic algorithms used for signature verification are secure and follow best practices.		✓	✓	

S6.2.B Standard Compliance (e.g., SIP-712)

Ref	Requirement	L1	L2	L3	SWE
S6.2.B1	Verify that ECDSA signature handling functions, such as ECDSA.recover and ECDSA.tryRecover, properly manage signature formats to prevent signature malleability, especially when handling both traditional 65-byte and SIP-2098 compact signatures.		✓	✓	

S6.3 Secure Random Number Generation

Control Objective

Implement best practices for secure random number generation to ensure unpredictability and resistance against manipulation.

S6.3.A Best Practices for Randomness

Ref	Requirement	L1	L2	L3	SWE
S6.3.A1	Ensure that random number generation follows best practices and uses secure sources of entropy.		✓	✓	
S6.3.A2	Verify that any random number generation is resistant to manipulation and prediction.		✓	✓	

S7. Arithmetic and Logic Security

Control Objective

Establish secure arithmetic and logic practices to prevent vulnerabilities such as overflow/underflow and ensure the integrity of calculations within smart contracts.

S7.1 Preventing Overflow/Underflow

Control Objective

Implement safe arithmetic practices to prevent overflow and underflow vulnerabilities that can compromise contract functionality and security.

S7.1.A Use of Safe Math Libraries

Ref	Requirement	L1	L2	L3	SWE
S7.1.A1	Verify that explicit type casting does not lead to overflow or underflow issues.		✓	✓	
S7.1.A2	Verify that integer arithmetic operations do not exceed their bounds to prevent integer overflow or underflow vulnerabilities.		✓	✓	
S7.1.A3	Ensure that operations involving time units and other expressions handle potential overflows correctly.		✓	✓	
S7.1.A4	Verify that division by zero is correctly handled and causes a transaction revert to prevent unexpected behavior.		✓	✓	
S7.1.A5	Ensure that variables are managed within their bounds to prevent reverts due to exceeding limits.		✓	✓	
S7.1.A6	Ensure that arithmetic operations within the unchecked{} block are carefully managed to prevent unintentional overflow or underflow.		✓	✓	
S7.1.A7	Confirm that inline assembly operations handle division by zero and overflow/underflow according to desired behavior and revert appropriately.		✓	✓	
S7.1.A8	Implement checks to handle cases where operations might introduce unintended precision issues or rounding errors.		✓	✓	

S7.1.B Fixed-Point Arithmetic

Ref	Requirement	L1	L2	L3	SWE
S7.1.B1	Verify that fixed-point arithmetic operations are performed safely to prevent overflow, underflow, and precision loss.		✓	✓	

S7.2 Arithmetic Integrity**Control Objective**

Ensure that all calculations and logical operations within the smart contract are performed correctly to maintain data integrity and prevent manipulation.

S7.2.A Secure Calculations and Logic

Ref	Requirement	L1	L2	L3	SWE
S7.2.A1	Ensure that price or rate calculations derived from asset balances are protected from manipulation, considering attack vectors like flash loans and donations.		✓	✓	
S7.2.A2	Ensure that the use of structs and arrays does not lead to data corruption or incorrect values due to storage encoding issues.		✓	✓	
S7.2.A3	Avoid operations that could lead to unintended data type conversions or precision loss by ensuring arithmetic operations are performed correctly.		✓	✓	
S7.2.A4	Enforce a minimum transaction amount to prevent attackers from clogging the network with zero amount or dust transactions.		✓	✓	
S7.2.A5	Validate that financial operations respect associative properties, ensuring consistent outcomes whether operations are performed in aggregate or iteratively.		✓	✓	
S7.2.A6	Implement proper rounding direction for calculations where accounting relies on user shares to avoid inaccuracies.		✓	✓	
S7.2.A7	Validate that inequalities and comparisons are correctly implemented to handle edge values appropriately.		✓	✓	

Ref	Requirement	L1	L2	L3	SWE
S7.2.A8	Ensure that abi.decode adheres to the type limits to avoid reverts due to overflow of target types.		✓	✓	
S7.2.A9	Ensure that logical operators such as ==, !=, &&, , and ! are used correctly, especially when test coverage may be limited.		✓	✓	

S7.2.B Precondition and Postcondition Checks

Ref	Requirement	L1	L2	L3	SWE
S7.2.B1	Ensure that multiplication is performed before division to maintain precision in calculations.		✓	✓	
S7.2.B2	Ensure that the request confirmation number is high enough to mitigate risks associated with chain re-orgs.		✓	✓	
S7.2.B3	Verify that off-by-one errors are avoided in loops and iterations, ensuring correct handling of list lengths and indexing.		✓	✓	
S7.2.B4	Verify that unsigned integers are not used to represent negative values, as this can lead to erroneous behavior.		✓	✓	
S7.2.B5	Verify that calculations with multiple terms handle all possible edge cases for min/max values to avoid errors.		✓	✓	

S8. Denial of Service (DoS)

Control Objective

Establish practices and mechanisms to prevent Denial of Service (DoS) attacks that can disrupt contract functionality and availability.

S8.1 Gas Limits

Control Objective

Ensure that contract design and function implementations are efficient in gas usage to mitigate risks associated with out-of-gas errors and related vulnerabilities.

S8.1.A Efficient Loop and Function Design

Ref	Requirement	L1	L2	L3	SWE
S8.1.A1	Ensure that contracts are protected against insufficient gas grieving attacks by carefully managing gas consumption in critical functions.		✓	✓	
S8.1.A2	Ensure that systems like the RocketDepositPool contract handle failures in functions like burn() gracefully.		✓	✓	
S8.1.A3	Verify that gas usage in functions and loops is efficient to avoid out-of-gas errors.		✓	✓	
S8.1.A4	Implement mechanisms to prevent denial of service attacks due to block gas limits, ensuring that transactions or operations do not exceed the gas limit constraints.		✓	✓	

S8.1.B Fallback Mechanisms

Ref	Requirement	L1	L2	L3	SWE
S8.1.B1	Ensure that try/catch blocks are provided with adequate gas to avoid failures and unexpected behavior in case of errors.		✓	✓	

S8.2 Resilience Against Resource Exhaustion

Control Objective

Implement strategies to protect contracts from resource exhaustion attacks that can lead to DoS scenarios.

S8.2.A Rate Limiting

Ref	Requirement	L1	L2	L3	SWE
S8.2.A1	Avoid using blocking mechanisms that could lead to a Denial-of-Service (DoS) attack.		✓	✓	
S8.2.A2	Protect against potential DoS in functions like <code>supportsSRC165InterfaceUnchecked()</code> by handling excessive data queries efficiently.		✓	✓	
S8.2.A3	Ensure that assertions do not lead to denial of service or unexpected contract reverts, especially in scenarios where conditions are not met.		✓	✓	
S8.2.A4	Verify that return values from external function calls are checked to prevent issues related to unchecked return values, which could lead to unexpected behavior.		✓	✓	
S8.2.A5	Ensure that contract functions are protected against denial of service due to unexpected reverts by handling all possible error conditions appropriately.		✓	✓	
S8.2.A6	Ensure that functions such as <code>supportsSRC165InterfaceUnchecked()</code> in <code>SRC165Checker.sol</code> handle large data queries efficiently to avoid excessive resource consumption.		✓	✓	

S9. Blockchain Data and State Management

Control Objective

Establish practices for effective management of blockchain data and state to ensure security, efficiency, and integrity of contract interactions.

S9.1 State Management

Control Objective

Ensure efficient and secure handling of state within smart contracts to prevent data corruption and unexpected behavior.

S9.1.A Efficient and Secure State Handling

Ref	Requirement	L1	L2	L3	SWE
S9.1.A1	Ensure that payable functions in contracts handle all SIL passed in msg.value and provide a mechanism for withdrawal to avoid SIL being locked in the contract.	✓	✓		
S9.1.A2	Verify that deleting a variable of a nested structure correctly resets all nested level fields to default values to avoid unexpected behavior.	✓	✓		
S9.1.A3	Verify that storage structs and arrays with types shorter than 32 bytes are handled correctly, avoiding data corruption when encoded directly from storage using the experimental ABIEncoderV2.	✓	✓		
S9.1.A4	Verify that storage arrays containing structs or other statically-sized arrays are properly read and encoded in external function calls to prevent data corruption.	✓	✓		
S9.1.A5	Ensure that copying bytes arrays from memory or calldata to storage handles empty arrays correctly, avoiding data corruption when the target array's length is increased subsequently without storing new data.	✓	✓		

S9.1.B State Channels

Ref	Requirement	L1	L2	L3	SWE
S9.1.B1	Verify that global state updates are correctly handled when working with memory copies to ensure accurate state management.		✓	✓	

S9.2 Data Privacy

Control Objective

Ensure that sensitive data within contracts is secured and that privacy measures are effectively implemented.

S9.2.A Ensuring Sensitive Data is Secure

Ref	Requirement	L1	L2	L3	SWE
S9.2.A1	Ensure that private data marked in contracts is protected from unauthorized access through blockchain analysis.		✓	✓	

S9.2.B Zero-Knowledge Proofs

Ref	Requirement	L1	L2	L3	SWE
S9.2.B1	Verify that zero-knowledge proofs are implemented to ensure privacy without revealing any underlying data.		✓	✓	
S9.2.B2	Validate the correctness of proof generation and verification processes to prevent any potential leaks or exploits.		✓	✓	
S9.2.B3	Ensure that zero-knowledge proofs are integrated seamlessly with the blockchain to maintain performance and security.		✓	✓	

S9.2.C Private Transactions

Ref	Requirement	L1	L2	L3	SWE
S9.2.C1	Verify that private transaction mechanisms (e.g., zk-SNARKs, zk-STARKs) are correctly implemented to ensure confidentiality of transaction details.		✓	✓	
S9.2.C2	Ensure that private transactions maintain the integrity and validity of the blockchain.		✓	✓	

S9.2.D Confidential Contracts

Ref	Requirement	L1	L2	L3	SWE
S9.2.D1	Verify that confidential contracts use cryptographic techniques to hide contract state and execution details from unauthorized parties.		✓	✓	
S9.2.D2	Ensure that only parties with appropriate permissions can access data within confidential contracts.		✓	✓	

S9.3 Event Logging

Control Objective

Implement transparent and secure logging practices to ensure traceability and detect unauthorized changes.

S9.3.A Transparent and Secure Logging Practices

Ref	Requirement	L1	L2	L3	SWE
S9.3.A1	Verify that events are emitted properly, especially for critical changes to ensure traceability and transparency.		✓	✓	
S9.3.A2	Verify that the contract's event logging correctly reflects critical changes to ensure transparency and traceability.		✓	✓	

S9.3.B Log Analysis

Ref	Requirement	L1	L2	L3	SWE
S9.3.B1	Implement tools and processes for analyzing event logs to detect anomalies or unauthorized changes.		✓	✓	
S9.3.B2	Set up alerts for unusual patterns or discrepancies in logged events.		✓	✓	

S9.4 Decentralized Storage

Control Objective

Ensure data integrity, security, and availability for data stored in decentralized storage solutions.

S9.4.A IPFS, Arweave

Ref	Requirement	L1	L2	L3	SWE
S9.4.A1	Ensure that data stored on decentralized platforms like IPFS or Arweave is encrypted and access-controlled.		✓	✓	
S9.4.A2	Implement mechanisms for data redundancy and backup to ensure data availability.		✓	✓	

S10. Gas Usage, Efficiency, and Limitations

Control Objective

Establish practices for optimizing gas usage and efficiency in smart contracts to minimize costs and enhance performance.

S10.1 Optimizing Gas Usage

Control Objective

Ensure gas consumption is minimized to promote cost-effective execution of smart contracts.

S10.1.A Understanding Gas Costs and Limits

Ref	Requirement	L1	L2	L3	SWE
S10.1.A1	Implement best practices for optimizing gas consumption to ensure cost-effective and efficient contract execution.		✓	✓	

S10.1.B Gas Estimation Tools

Ref	Requirement	L1	L2	L3	SWE
S10.1.B1	Verify that transaction confirmation numbers are chosen appropriately to mitigate risks related to chain re-orgs and ensure reliable contract operation.		✓	✓	

S10.2 Efficient Contract Design

Control Objective

Design contracts efficiently to enhance performance and reduce gas costs through optimal architecture.

S10.2.A Layer 2 Solutions

Ref	Requirement	L1	L2	L3	SWE
S10.2.A1	Explore and integrate Layer 2 scaling solutions (e.g., rollups, state channels) to improve transaction throughput and reduce gas costs.		✓	✓	
S10.2.A2	Verify the security and reliability of Layer 2 solutions before integration.		✓	✓	

S11. Component-Specific Security

Control Objective

Establish security practices and standards for various blockchain components to mitigate specific vulnerabilities associated with tokens, NFTs, vaults, and liquidity pools.

S11.1 Tokens (SRC20, SRC721, SRC1155)

Control Objective

Ensure secure implementation and management of token standards to prevent vulnerabilities.

S11.1.A Secure Implementation and Management

Ref	Requirement	L1	L2	L3	SWE
S11.1.A1	Verify that the totalSupply value is consistent during token minting operations, ensuring that callbacks do not result in incorrect values.	✓	✓		
S11.1.A2	Some tokens have multiple addresses associated with them, which can introduce vulnerabilities. Ensure all token addresses are managed and verified securely to avoid related risks.	✓	✓		
S11.1.A3	Verify that tokens handle zero amount transfers properly to prevent issues in integrations and operations.	✓	✓		
S11.1.A4	Verify that tokens handle zero amount transfers properly to prevent issues in integrations and operations.	✓	✓		
S11.1.A5	Some tokens revert on the transfer of a zero amount, which can cause issues in certain integrations and operations. Ensure compatibility with such tokens to avoid integration problems.	✓	✓		
S11.1.A6	Not all SRC20 tokens comply with the SIP20 standard; some may not return a boolean flag or revert on failure. Verify compliance with the SRC20 standard to avoid compatibility issues.	✓	✓		

S11.2 NFT Security

Control Objective

Implement best practices for non-fungible tokens to safeguard against vulnerabilities.

S11.2.A Best Practices for Non-Fungible Tokens

Ref	Requirement	L1	L2	L3	SWE
S11.2.A1	Implement standards and best practices for NFT creation, management, and transfer to prevent common vulnerabilities.		✓	✓	
S11.2.A2	Ensure proper metadata integrity and prevent unauthorized minting or transfers.		✓	✓	
S11.2.A3	Safeguard against potential exploits related to royalty payments or token burns.		✓	✓	

S11.3 Vaults

Control Objective

Ensure secure asset storage and management within vault systems.

S11.3.A Secure Asset Storage and Management

Ref	Requirement	L1	L2	L3	SWE
S11.3.A1	Address potential overhead issues associated with withdrawing stSIL or wstSIL, including queue times and withdrawal limits, to ensure smooth operations.		✓	✓	
S11.3.A2	Handle conversions between stSIL and wstSIL carefully to avoid potential issues due to the rebasing nature of stSIL.		✓	✓	

S11.4 Liquid Staking**Control Objective**

Ensure secure staking mechanisms to protect users' assets.

S11.4.A Secure Staking Mechanisms

Ref	Requirement	L1	L2	L3	SWE
S11.4.A1	Verify that mechanisms for detaching sfrxSIL from frxSIL are robust to prevent discrepancies and ensure accurate reward transfers, particularly when controlled by centralized entities.		✓	✓	
S11.4.A2	Monitor potential future changes in the sfrxSIL/SIL rate and ensure users are adequately forewarned to mitigate risks associated with rate fluctuations.		✓	✓	

S11.5 Liquidity Pools (AMMs)**Control Objective**

Establish security measures in automated market makers.

S11.5.A Security in Automated Market Makers

Ref	Requirement	L1	L2	L3	SWE
S11.5.A1	[WIP/Will be removed]				

S11.6 Uniswap V4 Hook**Control Objective**

Ensure secure integration and customization of Uniswap components.

S11.6.A Secure Integration and Customization

Ref	Requirement	L1	L2	L3	SWE
S11.6.A1	Verify the correct usage of Uniswap’s TickMath and FullMath libraries to ensure proper handling of unchecked arithmetic operations, adhering to version-specific Solidity considerations.		✓	✓	

Appendix A: Glossary

- **Access Control** –Mechanisms that restrict access to a system, application, or data to authorized users or entities. In smart contracts, access control is crucial for ensuring that only permitted users can perform sensitive actions.
- **Arithmetic Operations** –Basic mathematical operations (addition, subtraction, multiplication, division) performed in smart contracts. Proper handling of these operations is vital to prevent overflow and underflow vulnerabilities.
- **Audit** –A systematic examination of smart contracts to evaluate their security, functionality, and compliance with specified requirements. Audits help identify vulnerabilities and ensure adherence to best practices.
- **Bytecode** –The low-level code generated from Solidity (or other high-level languages) that is executed on the Sila Virtual Machine (SVM). Understanding bytecode is essential for analyzing contract behavior.
- **Denial of Service (DoS)** –An attack aimed at making a smart contract or service unavailable to its intended users, often by consuming excessive resources or exploiting vulnerabilities to cause failures in execution.
- **Fallback Function** –A default function in a smart contract that is executed when a contract receives SIL without any accompanying data or when a function that doesn't exist is called. Proper design of fallback functions is important to prevent security issues.
- **Gas** –A unit that measures the computational work required to execute operations on the Sila blockchain. Gas fees incentivize miners and limit the complexity of transactions.
- **Gas Limit** –The maximum amount of gas a user is willing to pay for a transaction, impacting the transaction's likelihood of being included in a block.
- **Layer 2 Solutions** –Technologies built on top of existing blockchains to enhance scalability and reduce transaction costs. Examples include state channels and rollups, which alleviate congestion on the main chain.
- **Minting** –The process of creating new tokens or assets and assigning them to a specified address. This operation must be carefully managed to ensure compliance with token standards.
- **Non-Fungible Token (NFT)** –A unique digital asset that represents ownership of a specific item or piece of content, distinguished by its distinct characteristics, making it irreplaceable.
- **Overflows and Underflows** –Vulnerabilities that occur when arithmetic operations exceed the maximum or minimum value of a data type, leading to unexpected behavior. Safe math libraries help prevent these issues.
- **Reentrancy** –A vulnerability where a function makes an external call to another contract before completing its execution, potentially allowing the second contract to manipulate the state of the first contract before it finishes processing.

- **Security Audit** –A comprehensive review and evaluation of a smart contract’s code to identify vulnerabilities, inefficiencies, and compliance with best practices.
- **Smart Contract** –A self-executing contract with the terms of the agreement directly written into code and deployed on a blockchain. Smart contracts automate execution without the need for intermediaries.
- **Token Standard** –Specifications that define how tokens should function on a blockchain. Common standards include SRC20 for fungible tokens, SRC721 for non-fungible tokens, and SRC1155 for multi-token standards.
- **Transaction Confirmation** –The process by which a transaction is validated and recorded on the blockchain. A transaction must be confirmed by miners to be considered final and irreversible.
- **Vulnerability** –A weakness in a smart contract that can be exploited by an attacker, leading to unauthorized access, data breaches, or financial loss.
- **Whitelisting** –A security practice where specific addresses or entities are granted permission to interact with a contract, enhancing access control and mitigating potential attacks.
- **Zero-Knowledge Proofs** –Cryptographic methods that allow one party to prove to another that they know a value without revealing the value itself. This is used to enhance privacy in blockchain transactions.
- **Audit Trail** –A chronological record that tracks the sequence of activities and changes made to a smart contract, providing transparency and accountability.
- **SRC Standards** –Sila Request for Comments; a series of technical documents that provide guidelines and specifications for the development of smart contracts and tokens on the Sila blockchain.
- **Decentralized Finance (DeFi)** –A financial ecosystem that operates without central authorities, using smart contracts on blockchains to provide financial services like lending, borrowing, and trading.
- **Oracle** –A third-party service that provides external data to smart contracts, enabling them to interact with real-world information such as prices, events, or weather data.
- **Tokenomics** –The study of the economic model and incentive structures behind cryptocurrencies and tokens, including supply, demand, and the distribution of tokens.
- **Gas Optimization** –Techniques and practices aimed at reducing the gas consumption of smart contracts, thereby lowering transaction costs and improving efficiency.
- **Atomic Swap** –A smart contract technology that enables the exchange of one cryptocurrency for another without the need for a trusted third party, ensuring security and trustlessness.
- **Cryptographic Hash Function** –A mathematical algorithm that transforms input data into a fixed-size string of characters, which is unique to each unique input. This function is crucial for ensuring data integrity in blockchain.

- **State Machine** –A model that represents the state of a smart contract and its transitions, allowing for tracking of the current state and the possible changes based on events and actions.
- **Gas Refund** –A mechanism that allows users to recover some of the gas fees spent on certain operations, particularly those that free up storage space on the blockchain.
- **Contract Upgradeability** –The ability to modify or replace a smart contract after its deployment to fix bugs or add new features, often implemented through proxy patterns.
- **Security Vulnerability Disclosure** –A responsible disclosure process where security researchers report vulnerabilities found in smart contracts to the developers, allowing them to address the issues before public knowledge.
- **Interoperability** –The capability of different blockchain networks and smart contracts to communicate and interact with each other, enabling seamless integration of services and assets across platforms.

Appendix B: References

The following OWASP projects are most likely to be useful to users/adopters of this standard:

OWASP Core Projects

1. OWASP Top 10 Project
2. OWASP Smart Contract Top 10 Project